

Ingeniería del Software II

Práctica 6: Refactorizaciones

Informe de mantenimiento preventivo y análisis de métricas

Alumno	Javier Lamas Tabuenca
Asignatura	Ingeniería del Software II
Curso	2025/2026
Repositorio / rama	IS_2526 / practica6

Índice

1. Introducción y objetivos
2. Descripción del sistema heredado
3. Metodología de trabajo
4. Situación inicial del sistema
5. Refactorizaciones aplicadas
6. Situación final del sistema
7. Comparación y análisis de mejora
8. Pruebas de regresión y generación del JAR
9. Conclusiones
10. Referencias

1. Introducción y objetivos

El objetivo de esta práctica es aplicar un proceso de refactorización sobre un sistema heredado de gestión de transportes, manteniendo su comportamiento externo y mejorando su estructura interna. Para ello se parte del código original proporcionado, se organiza en un proyecto Maven, se calculan métricas de calidad y, posteriormente, se crea un segundo proyecto Maven independiente sobre el que se aplican las refactorizaciones.

La práctica se centra especialmente en dos aspectos: la aplicación de refactorizaciones como técnica de mantenimiento preventivo y el cálculo de métricas de producto como WMC, WMCn, CCOg, CCogn, CBO, DIT y NOC. También se han ejecutado pruebas de regresión mediante JUnit y Maven para comprobar que las modificaciones no introducen errores.

2. Descripción del sistema heredado

El sistema permite a una empresa de transporte gestionar conductores y transportes realizados. Cada conductor se identifica mediante su DNI y tiene asociados sus datos personales y una lista de transportes. Cuando un conductor finaliza un transporte, este se añade a su historial.

El sueldo de cada conductor se calcula con una cantidad fija y una parte variable:

- Sueldo base fijo: 700 euros.
- Extra básico: 5 euros por cada hora conducida.
- Transporte de personas: 0,5 euros por hora si no es colectivo, y 1 euro por hora si es colectivo.
- Transporte de mercancías: 2 euros por tonelada transportada.
- Transporte de mercancías peligrosas: igual que mercancías más 50 euros fijos por transporte.

3. Metodología de trabajo

1. Se organizó el código original en un proyecto Maven llamado transportes-original.
2. Se ejecutaron los tests JUnit del proyecto original mediante mvn clean test.
3. Se calcularon las métricas iniciales y se documentó el cálculo de WMC y CCOg mediante comentarios en el código.
4. Se creó un segundo proyecto Maven independiente llamado transportes-refactorizado, con artifactId propio.
5. Se aplicaron refactorizaciones sobre el segundo proyecto, sin modificar el proyecto original.
6. Después de los cambios se ejecutaron de nuevo las pruebas de regresión.
7. Finalmente se generó un JAR ejecutable del proyecto refactorizado.

4. Situación inicial del sistema

En la versión inicial, la clase Conductor contenía la lógica de cálculo del sueldo y decidía el comportamiento según la categoría del transporte. La clase Transporte almacenaba una CategoriaTransporte y un atributo valor, cuyo significado dependía del tipo de transporte: número de personas en transportes de personas o toneladas en transportes de mercancías. Esta solución funciona, pero concentra demasiada lógica de negocio en Conductor y dificulta la extensión del sistema.

4.1 Diagrama de clases inicial

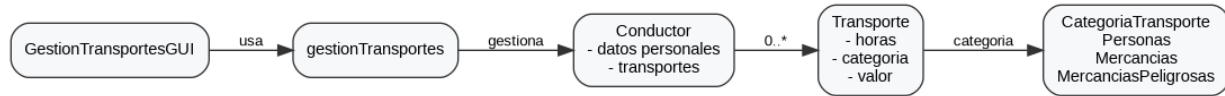


Figura 1. Diagrama de clases inicial del sistema heredado.

4.2 Problemas detectados

Problema	Descripción
Abuso de condicionales	El cálculo del sueldo dependía de un switch sobre CategoriaTransporte dentro de Conductor.sueldo().
Baja extensibilidad	Añadir un nuevo tipo de transporte obligaría a modificar Conductor.sueldo(), aumentando el riesgo de introducir errores.
Atributo ambiguo	El atributo valor de Transporte representaba personas o toneladas según la categoría.
Números mágicos	Valores como 700, 5, 0.5, 1, 2, 10 y 50 aparecían directamente en el código.
Nombre de clase mejorable	La clase gestionTransportes comenzaba con minúscula, incumpliendo la convención habitual de nombres de clase en Java.

4.3 Métricas iniciales

Para el cálculo se ha seguido el siguiente criterio: los constructores se consideran métodos; WMC es la suma de las complejidades ciclomáticas de los métodos; WMCn divide WMC entre el número de métodos; CCog divide CCog entre el número de métodos; para DIT se ignora java.lang.Object; y para CBO se contabilizan las dependencias entre clases propias del sistema.

Clase	Métodos	WMC	WMCn	CCog	CCogn
CategoriaTransporte	0	0	0.00	0	0.00
Transporte	5	9	1.80	4	0.80
Conductor	9	18	2.00	9	1.00
gestionTransportes	3	6	2.00	4	1.33
GestionTransportesGUI	2	18	9.00	34	17.00

Clase	CBO	Clases que contribuyen al CBO	DIT	NOC
CategoriaTransporte	0	Ninguna	0	0
Transporte	1	CategoriaTransporte	0	0
Conductor	1	Transporte	0	0
gestionTransportes	1	Conductor	0	0
GestionTransportesGUI	4	gestionTransportes, Conductor, Transporte, CategoriaTransporte	0	0

5. Refactorizaciones aplicadas

Las refactorizaciones se aplicaron sobre el proyecto transportes-refactorizado, manteniendo separado el proyecto original. El orden seguido fue el siguiente:

Orden	Refactorización	Aplicación en la práctica
1	Rename Class	Se cambió gestionTransportes por GestionTransportes para ajustarlo a la convención de nombres de Java.
2	Replace Magic Number with Symbolic Constant	Se sustituyeron literales de negocio por constantes con nombre: SUELDO_BASE, EXTRA_BASE_HORA, EXTRA_POR_TONELADA, EXTRA_MERCANCIAS_PELIGROSAS, LIMITE_COLECTIVO, etc.
3	Extract SuperClass/SubClass	Transporte pasó a ser una clase abstracta común y se crearon las subclases TransportePersonas, TransporteMercancias y TransporteMercanciasPeligrosas.
4	Move Method	El cálculo del extra de cada transporte se movió desde Conductor.sueldo() hacia la jerarquía de Transporte.
5	Replace Conditional with Polymorphism	Se eliminó el switch de Conductor.sueldo() y cada subtipo de transporte implementa su propia parte del cálculo mediante extraAdicional().
6	Adaptación de pruebas	Los tests se adaptaron para instanciar las nuevas subclases sin cambiar los escenarios funcionales comprobados.

5.1 Nueva responsabilidad de Transporte

En la solución final, Transporte contiene la parte común del cálculo del extra: las horas conducidas multiplicadas por el extra base por hora. Cada subclase implementa extraAdicional(), de modo que la variación de comportamiento queda localizada en el subtipo correspondiente.

```
public double extra() {
    return horas * EXTRA_BASE_HORA + extraAdicional();
}
```

```
protected abstract double extraAdicional();
```

5.2 Simplificación de Conductor.sueldo()

El método sueldo() deja de conocer las categorías concretas de transporte. Ahora solo recorre la lista de transportes y delega el cálculo en cada objeto.

```
public double sueldo() {
    double sueldoTransportes = 0;
    for (Transporte t : transportes) {
        sueldoTransportes += t.extra();
    }
}
```

```

return SUELDO_BASE + sueldoTransportes;
}

```

6. Situación final del sistema

Tras la refactorización, el diseño se basa en una jerarquía de transportes. La clase Conductor mantiene únicamente la responsabilidad de almacenar sus datos y sus transportes, mientras que cada tipo concreto de transporte sabe calcular su extra específico. La categoría deja de ser necesaria como enumerado para decidir el comportamiento del cálculo.

6.1 Diagrama de clases final

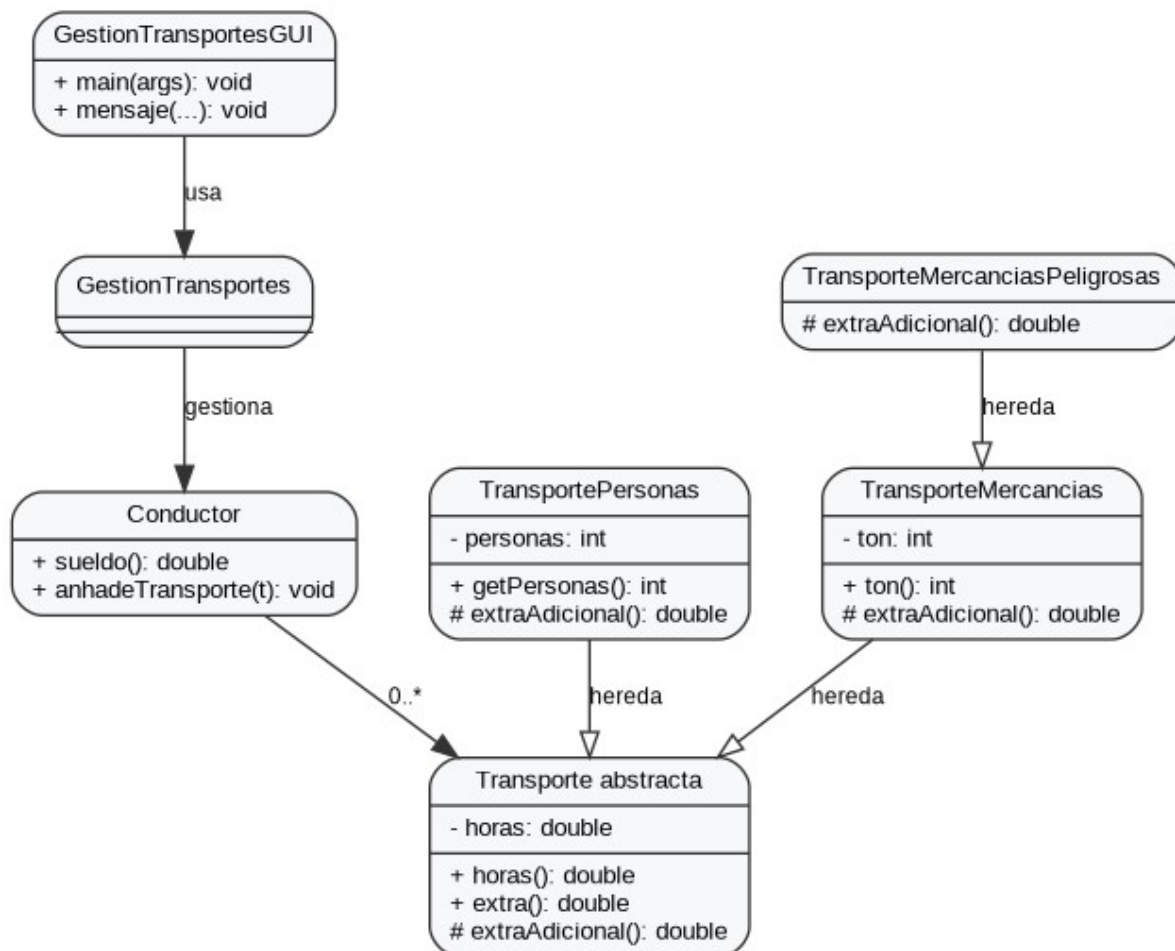


Figura 2. Diagrama de clases final tras aplicar las refactorizaciones.

6.2 Métricas finales

Clase	Métodos	WMC	WMCn	CCog	CCogn
Transporte	3	4	1.33	1	0.33
TransportePersonas	3	5	1.67	2	0.67
TransporteMercancias	3	4	1.33	1	0.33
TransporteMercanciasPeligrosas	2	2	1.00	0	0.00
Conductor	9	14	1.56	3	0.33
GestionTransportes	3	6	2.00	4	1.33
GestionTransportesGUI	2	18	9.00	34	17.00

Clase	CBO	Clases que contribuyen al CBO	DIT	NOC
Transporte	0	Ninguna	0	2
TransportePersonas	1	Transporte	1	0
TransporteMercancias	1	Transporte	1	1
TransporteMercanciasPeligrosas	1	TransporteMercancias	2	0
Conductor	1	Transporte	0	0
GestionTransportes	1	Conductor	0	0
GestionTransportesGUI	6	GestionTransportes, Conductor, Transporte, TransportePersonas, TransporteMercancias, TransporteMercanciasPeligrosas	0	0

7. Comparación y análisis de mejora

Elemento	Métrica	Inicial	Final	Interpretación
Conductor	WMC	18	14	Baja 4 puntos al eliminar el switch y delegar el cálculo en Transporte.
Conductor	CCog	9	3	Baja notablemente porque sueldo() deja de tener switch e if anidados.
Conductor.sueldo()	CC	6	2	Pasa de for + switch + if a un único for con delegación polimórfica.
Conductor.sueldo()	CCog	7	1	El método es mucho más directo y comprensible.
Transporte	NOC	0	2	Aumenta porque ahora es una superclase con hijos directos.
TransporteMercanciasPeligrosas	DIT	-	2	Aumenta por heredar de TransporteMercancias, reutilizando el cálculo común por tonelada.

El cambio más importante se aprecia en Conductor. En la versión inicial, sueldo() concentraba la regla general de sueldo y todas las reglas particulares de cada tipo de transporte. Esto aumentaba la complejidad y obligaba a modificar Conductor cada vez que apareciera un nuevo tipo. En la versión final, Conductor queda cerrado frente a modificaciones de tipos concretos de transporte y abierto a extensión mediante nuevas subclases de Transporte.

El aumento de DIT y NOC no debe interpretarse como un empeoramiento automático. En este caso, el incremento de herencia responde a una decisión de diseño: se introduce polimorfismo para eliminar lógica condicional y repartir responsabilidades. La mejora se refleja especialmente en la reducción de WMC y CCog de Conductor, que era la clase con mayor carga de lógica de negocio.

8. Pruebas de regresión y generación del JAR

Las pruebas se ejecutaron con Maven mediante el comando `mvn clean test` tanto en el proyecto original como en el refactorizado. Los casos de prueba del proyecto refactorizado mantienen los mismos escenarios funcionales, aunque se adaptó su implementación para instanciar `TransportePersonas`, `TransporteMercancias` y `TransporteMercanciasPeligrosas` en lugar de crear directamente `Transporte` con una `CategoriaTransporte`.

Ejecuciones realizadas:

- `transportes-original: mvn clean test -> BUILD SUCCESS.`
- `transportes-refactorizado: mvn clean test -> BUILD SUCCESS.`
- `transportes-refactorizado: mvn clean package -> generación correcta del JAR ejecutable.`
- Prueba manual del JAR: `java -jar transportes-refactorizado-1.0-SNAPSHOT.jar -> la aplicación se ejecuta correctamente.`

El uso de pruebas de regresión es especialmente importante en esta práctica, porque el comportamiento externo del sistema no debe cambiar aunque se modifique la estructura interna. En este caso, los tests validan tanto la construcción de conductores como el cálculo del sueldo para transportes de personas, mercancías y mercancías peligrosas.

9. Conclusiones

La refactorización ha mejorado la mantenibilidad del sistema sin modificar su comportamiento funcional. La mejora principal consiste en sustituir una solución basada en condicionales por una solución polimórfica. De esta forma, las reglas específicas de cada transporte quedan encapsuladas en su propia clase.

El resultado es un diseño más claro, más fácil de ampliar y menos propenso a errores. Si en el futuro se añadiera un nuevo tipo de transporte, no sería necesario modificar `Conductor.sueldo()`; bastaría con crear una nueva subclase de `Transporte` e implementar `extraAdicional()`.

Las métricas respaldan esta mejora: `Conductor` reduce su WMC y su complejidad cognitiva, especialmente en el método `sueldo()`. Aunque aparecen nuevas clases y una jerarquía de herencia, este cambio está justificado porque mejora la separación de responsabilidades y permite eliminar el switch de la lógica de negocio.

10. Referencias

- Enunciado de la Práctica 6: Refactorizaciones, Ingeniería del Software II, Universidad de Cantabria.
- Transparencias del Tema 6: Mantenimiento de sistemas software y métricas de mantenibilidad.
- Transparencias del Tema 6: Refactorizaciones más comunes.
- Seminario de automatización de pruebas en Java: JUnit, EclEmma y FEST.